There really is only one case where Django-based authentication should be used: when the API is for a web-based interface. If your API is there to support an SPA or provide data to libraries like Vue.js, then the redirect isn't a problem. You can capture the redirect in your code and send the user to the login page. To implement this kind of authentication, simply import `django_auth` from `ninja.security` and pass it as the value to the `auth` argument in an endpoint function or the `NinjaAPI` class.

If Django-based authentication isn't the answer, then what is? A common mechanism out there is to provide a key. Keys can be simple, a global string which all API consumers know, or more complex, a value stored in the database and validated. For public-facing APIs, you're going to want a key management system, so you can revoke keys from problematic users. If you're just using the API for your own tools, you can take a simpler approach. Be careful with keys; they are essentially passwords. If you're passing them around in the clear, someone might see them and then gain access to your API. For simplicity, I'm going to ignore this advice here and just use a hardcoded global value.

Getting key management right

Like password management, key management can be tricky. If you're building a public API, you should do some research on key management first. A good place to start is freeCodeCamp's website: https://mng.bz/maan.

The major cloud providers, like AWS (https://aws.amazon.com/what-is/api-key/) and Google Cloud (https://mng.bz/5lO4), both have in-depth documentation on how they handle API keys at scale, to give you some ideas as well.

When you authenticate with Django, it sets a cookie in your browser and uses that to validate who you are in future calls. This requires managing a user's session. For an API, it is far simpler to require the key to be included in every single call. HTTP supports metadata getting attached to a method through the use of a header. HTTP has its own headers but reserves anything that begins with *X-* as a namespace to be used by developers. Ninja provides the `APIKeyHeader` class, which you extend to create a key-based authentication mechanism.

Your subclass of `APIKeyHeader` needs two things: the name of the HTTP header to use for the key and an `.authenticate()` method. The authentication method returns the key to indicate success or `None` for failure. You provide an instance of your subclass as the `auth` argument in the places you wish authentication to happen. In the following examples, I'm going to put it on the endpoints. There should only be one instance of the `APIKeyHeader` subclass, so it is useful to put it in a file separate from api.py files in your apps. It doesn't really matter where it goes; it just needs to be importable. Create RiffMates/api_auth.py, and add the code in the following listing.